ALLIANCE UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

DATABASE MANAGEMENT SYSTEM

ASSIGNMENT 8

SQL QUERIES AND AGGREGATE FUNCTIONS

Hotel Management System

Particular Details	**Information**
Student Name	A Praisy Emberline
Programme	BCA – Artificial Intelligence & Machine Learning
Subject	Database Management System
University	Alliance University
Academic Year	2nd Year

Introduction

SQL queries retrieve, filter, sort, and analyse records stored in a database. WHERE filters rows, ORDER BY sorts results, GROUP BY creates groups, and HAVING filters groups. Aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX() perform calculations. The following queries use the sample Hotel Management System data from Assignment 7.

1. Filtering Records Using WHERE

The WHERE clause returns only rows that satisfy a condition.

```
SELECT room_id, room_number, room_status
FROM Room
WHERE room_status = 'Available';
```

Expected output:

room_id	room_number	room_status
201	101	Available
203	301	Available

This query displays rooms currently marked as Available.

2. Sorting Records Using ORDER BY

ORDER BY arranges rows in ascending (ASC) or descending (DESC) order.

```
SELECT type_name, price_per_night
FROM Room_Type
ORDER BY price_per_night DESC;
```

Expected output:

type_name	price_per_night
Suite	6500.00
Deluxe	4000.00
Standard	2500.00

The room types are displayed from the highest to the lowest nightly price.

3. Grouping Records Using GROUP BY

GROUP BY groups rows that share the same value. It is commonly used with aggregate functions.

```
SELECT reservation_status, COUNT(*) AS total_reservations
FROM Reservation
GROUP BY reservation_status
ORDER BY reservation_status;
```

Expected output:

reservation_status	total_reservations
Checked-in	1
Confirmed	2

The query counts reservations in each status group.

4. Filtering Grouped Results Using HAVING

HAVING filters groups after GROUP BY has formed them.

```
SELECT room_type_id, COUNT(*) AS total_rooms
FROM Room
GROUP BY room_type_id
HAVING COUNT(*) >= 1
ORDER BY room_type_id;
```

Expected output:

room_type_id	total_rooms
1	1
2	1
3	1

The query displays room types that have at least one room. In this sample, each room type has one room.

5. Aggregate Functions

A. COUNT()

```
SELECT COUNT(*) AS total_guests
FROM Guest;
```

Expected output: total_guests = 3. COUNT() returns the number of guest records.

B. SUM()

```
SELECT SUM(amount) AS total_payment_amount
FROM Payment;
```

Expected output: total_payment_amount = 27000.00. This sum includes the pending payment because the query counts all rows.

C. AVG()

```
SELECT AVG(price_per_night) AS average_nightly_price
FROM Room_Type;
```

Expected output: average_nightly_price = 4333.333333. The average is calculated from 2500.00, 4000.00, and 6500.00.

D. MIN()

```
SELECT MIN(service_price) AS minimum_service_price
FROM Service;
```

Expected output: minimum_service_price = 300.00, the lowest listed service price.

E. MAX()

```
SELECT MAX(price_per_night) AS maximum_nightly_price
FROM Room_Type;
```

Expected output: maximum_nightly_price = 6500.00, the highest listed room-type price.

6. Additional Query: Reservations for One Guest

```
SELECT reservation_id, check_in_date, check_out_date, reservation_status
FROM Reservation
WHERE guest_id = 101
ORDER BY check_in_date;
```

Expected output: reservation 1001, check-in 2026-08-25, check-out 2026-08-27, status Checked-in.

Conclusion

Basic SQL queries and aggregate functions were applied to the Hotel Management System. WHERE filtered room records, ORDER BY sorted room prices, GROUP BY counted reservation statuses, and HAVING filtered groups. COUNT(), SUM(), AVG(), MIN(), and MAX() calculated useful information from the sample data.